

Enhancing Highway Safety with LSTM-based Vehicle Intention Prediction (HVIP)

Lujina Amer^{*†} and Catherine M. Elias^{*†} 

^{*}C-DRiVeS Lab: Cognitive Driving Research in Vehicular Systems, Cairo, Egypt

Email: cdrives.researchlab@gmail.com

[†]Computer Science and Engineering Department - Faculty of Media Engineering and Technology

German University in Cairo, Egypt

Email: lujinaabdullah50@gmail.com, catherine.elias@ieee.org

Abstract—Predicting vehicles’ motion on highways has become crucial for enhancing road safety and traffic flow. Deep learning, which reached exceptional results in various applications, is now the leading approach for vehicle motion prediction. This work presents a deep learning-based model using Long Short-Term Memory (LSTM) networks to classify vehicle intentions into lane-keeping, left lane-changing, and right lane-changing. Utilizing the PREVENTION dataset, which provides naturalistic driving data, a sequence of centre points, longitudinal distances, lateral distances and yaw angles of a vehicle has been extracted to train the model to predict lane changes effectively. The model consists of three LSTM layers, a dense output layer and a drop out added between layers to prevent overfitting. Experiments focusing on optimizing the model parameters, learning rate, sequence length, and batch size, were conducted to determine their impact on prediction effectiveness. The best-performing model showed significant results in lane changing prediction, achieving an accuracy of 84%, precision of 89%, and recall of 82%. Future work will aim to train the model on more complex features, to account for vehicle inter-dependencies and interactions, and test model on various highway scenarios for a more reliable system.

Keywords: Intention Prediction, Lane-Changing, Autonomous Vehicle, Long short-term memory (LSTM), Prevention Dataset.

I. INTRODUCTION AND RELATED WORKS

With advancements in the automotive technology and analysis of past crash data, newer vehicles are built with more safety features to protect the road users and reduce the damages of an accident. According to the World Health Organization (WHO) [1], the number of traffic deaths have seen a 5% drop with 1.19 deaths in 2021 compared to 1.25 in 2010. This slight decrease shows that the efforts towards road safety are effectively reducing the risks, in spite of the global population increasing by about a million and the number of vehicles worldwide doubling. However, this decrease is still not enough to meet the need of the global roads safety. Research [2] found that 75% of traffic accidents occur because of the driver making a wrong lane change decision. Lane changing is an action performed by the vehicle driver to move from the current lane to an adjacent lane after examining the adjacent lanes conditions and ensuring the safety of the lane change. Recognizing a lane change intention correctly can prevent an accident and affect the traffic flow. As shown by [3], only 40% of drivers tend to use turn signals before starting a lane change which makes relying on visual cues for recognizing a lane change

insufficient. This shows the critical need to develop a reliable intention prediction system for future autonomous vehicles. Such technology will improve the perception of the traffic context and help in identifying the intentions of surrounding drivers, which can prevent accidents and positively affect the traffic flow. Understanding the intentions of surrounding vehicles in a dynamic environment can be challenging and affected by several factors, such as the interaction between multiple road agents and how their actions might influence one another. Moreover, perceiving the world through sensors makes the system liable to sensor occlusions causing a single moment’s sensor reading to become unreliable. Therefore, the model’s capability to analyse the temporal relationship between sensor readings will affect the accuracy of predictions [4].

This topic has been the focus of many researchers, and several approaches have been proposed for predicting the motion of the surrounding vehicles. [5] has classified the vehicle motion model approaches into three different levels which are Physics-based, maneuver-based and interaction-aware motion models, and provided examples of each type. Physics-based models are the simplest among the three types, and can be categorized into dynamic models that describe vehicle state by the various forces that impact its’ motion, or kinematic models which ignore these forces and instead focus on the mathematical relationships between parameters such as position and velocity to describe the vehicle state [5]. However, these models are limited to short-term motion prediction. In contrast, maneuver-based motion models are introduced as being reliable in long term predictions and not limited to short term ones. These models assume that a vehicle’s motion at any given moment corresponds to one maneuver from a set of possible maneuvers, and that its’ movement is independent of the other surrounding vehicles. By using a maneuver-based motion model, predicting a trajectory becomes simpler compared to older approaches like prototype trajectory approaches. One issue with the prototype trajectory approach is that it requires a very large amount of training trajectory data to be able to represent all the possible motion patterns of a vehicles. A more generalized approach would be to use maneuver-based trajectory prediction, which first estimates the maneuver intention by utilizing higher-level features, then matches the upcoming vehicle’s motion to a potential trajectory for the intended

maneuver [5]. Rapidly-Exploring Random Tree (RRT) [6], Gaussian Processes (GP) [7], and Stochastic Reachable Sets [8] are chosen for maneuver-based trajectory prediction due to their ability to account for the uncertainties in the vehicle's state and the unpredictability of maneuvers performed and its execution [5]. Although maneuver-based models can produce promising results in some scenarios, the inter-dependencies between vehicles still needs to be addressed, as the movement of one vehicle may affect another. This shows the necessity of an interaction-aware motion model that represents vehicles as dynamic entities influencing each other [5]. [9] modeled the multiple interactions between vehicles using Coupled HMMs (CHMMs). However, due to the increasing complexity with the growing number of dependencies in complex traffic scenarios, a simpler asymmetric CHMM model [10] was proposed, which only considers the impact of the surrounding traffic on the target vehicle. Moreover, [11] utilized traffic rules in prediction of vehicles trajectories through an interaction aware motion model. Lately, deep learning has become the leading approach in machine learning achieving remarkable results across various applications including audio and speech processing, visual data processing, natural language processing (NLP) [12]. Following the appearance of several deep learning-based models for vehicle motion prediction, [13] further classified motion models, excluding physics based models, which are no longer state-of-art, into three classifications based on input representation, output type, and prediction method. A recent work [14] proposed two distinct deep learning models for predicting lane changes: a Recurrent Neural Network (RNN) model and a Long Short-Term Memory (LSTM) model. A sequence of changing vehicles centroids was fed to both models. Experimental results showed that LSTM model outperformed the RNN in long term prediction, mainly due to the RNN's vanishing gradient problem. Moreover, the influence of learning rate, dropout, and sequence length values on performance was explored. Lane changes were also predicted in a different research by [15], where a deep learning-based Multi-LSTM network was proposed. The first LSTM layer was used to extract characteristics and the second LSTM layer was used for the predicting the intention. The affect of number of hidden layers and length of data sample on accuracy was also investigated.

In this work, a deep learning-based model is introduced to classify the intention of highway vehicles into three classes which are lane-keeping, left lane-changing, and right lane-changing. The PREVENTION dataset was used to extract the input features utilizing the different driving scenarios and the naturalistic driving data this dataset has to offer. The PREVENTION dataset is relatively new and not widely used in existing studies, therefore this work will share some potential features within this dataset for vehicle motion prediction and their extraction methods. Fig 1. shows the pipeline for the vehicle intention prediction process.

The rest of the paper is organized as follows. Section II. introduces the PREVENTION dataset and features used for model training. Section III. describes the LSTM model

architecture, the model parameters and testing. Section IV. explores the affect of tuning model parameters. Finally, Section VI. concludes the paper and suggests future work.

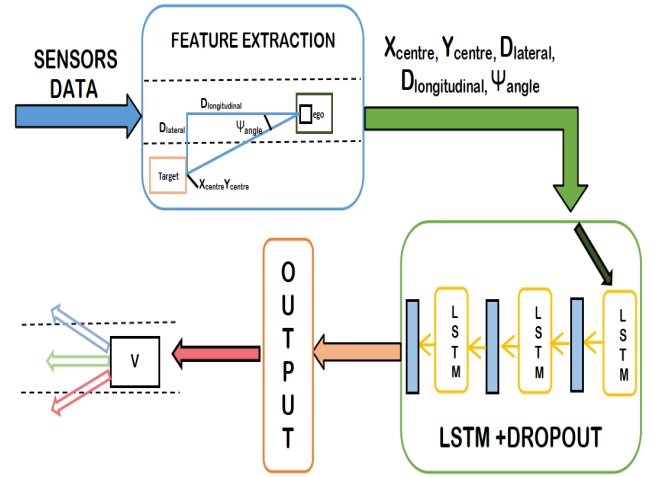


Fig. 1: Pipeline used for our vehicle intention prediction process, capturing each step from data processing to prediction output

II. PREVENTION DATASET

The dataset utilized in this work for feature extraction and model training is the PREVENTION dataset [16]. This dataset was collected in 2019 by the members of the Computer Engineering Department, Universidad de Alcala, Alcala de Henares, Spain, to support the ongoing research in vehicle intention and trajectory prediction. The Dataset website can be publicly accessed at <https://prevention-dataset.uah.es/>. The website features five recordings from five different days and three different areas, with a total of more than 6 hours of driving and covering approximately 500 kilometers. Each recording may contain one or more drives, where each drive includes calibration data, raw and processed sensors data. A C/C++ code is also offered in the website to help import and utilize the data provided.

A. Dataset Sensors

This dataset was collected using six time-synced and cross-calibrated on-board sensors: three radars, one LiDAR, and two high-resolution cameras. Readings coming from these 6 sensors are combined with data from GPS, IMU, and CAN bus. The cameras used in this dataset are two Grasshopper3 cameras covering a Field Of View (FOV) of 48° in the front and the back with a resolution of 1920×1200. The LiDAR is a Velodyne HDL-32E which has a vertical FOV that range from +10° to -30°. The range of the LiDAR is up to 100 m with an error lower than 2 cm. For the purpose of this work, we will only focus on data obtained from cameras and LiDAR.

B. Data Structure and Analysis

In **processed_data** folder, processed data obtained from all sensors are organised into different folders. In this work, we are only interested in **detection_camera1** and **detection_cloud** folders. Inside **detection_camera1** folder **detections.txt** can be found, which lists all detections in the format *[frame id class xi yi xf yf conf n nContourValues]*. To obtain these camera detections, the actors of each scene were automatically segmented and divided into six classes: cars, trucks, buses, motorcycles, bicycles, and pedestrians. The raw outputs are provided as temporally integrated bounding boxes and contour values corresponding to detected class object [16]. Our primary concern is the vehicle classes, therefore, non-vehicle classes were excluded. The **detection_filtered.txt** contains only the detections that meet a minimum confidence value of 0.5, and **detection_tracked.txt** gives each detection a unique id. The **detection_camera1** folder also contains **lane_changes.txt** in the form of *[ID ID_m LC_type f0 f1 f2 blinker]* where *ID_m* is the unique ID of the vehicle performing the lane change, *LC_type* is 3 for a left lane change and 4 for a right lane change, *f0* is frame in which the lane change starts, *f1* is the frame in which the centre of the vehicle performing lane change is between the two lanes and *f2* is the frame in which the lane change ends. The *blinker* shows whether a blinker was used by the vehicle with 0 indicating no blinker used and 1 indicating blinker used. Moving to **detection_cloud** folder, a **ground_coefficients.txt** file can be found which holds the parameters of a scalar plane equation based on the LiDAR reference system, arranged as *[frame A B C D]* and expressed in m^{-1} as shown in equation 1.

$$Ax + By + Cz + D = 0 \quad (1)$$

C. Feature Extraction

Using data from **detection_camera1** and **detection_cloud** folders, five features were extracted. The input features used to train the model are:

- $(X_{\text{centre}}, Y_{\text{centre}})$: the centre point of the target vehicle on the image. This feature was highly inspired by [1].
- D_{lateral} : the lateral distance of the target vehicle from the ego vehicle's LiDAR as shown in Fig. 1.
- $D_{\text{longitudinal}}$: the longitudinal distance of the target vehicle from the ego vehicle's LiDAR as shown in Fig. 1.
- ψ_{yaw} : the angle between the ego vehicle's LiDAR and the target vehicle as shown in Fig. 1.

Ego vehicle is the vehicle that contains the on-board sensors, while target vehicle is the vehicle, whose motion we are trying to predict.

Extracting the centre points $(X_{\text{centre}}, Y_{\text{centre}})$ was straightforward, thanks to the **detection_tracked.txt** provided by the dataset. By extracting the coordinates (xi, yi) at the top-left and (xf, yf) at the bottom-right corners for each vehicle bounding box, the center point was calculated. Each centre point was saved along with the unique vehicle ID provided by

detection_tracked.txt and the frame it was retrieved from to be used later on in creating the input sequences.

Calculating the lateral distance D_{lateral} , the longitudinal distance $D_{\text{longitudinal}}$ and the angle ψ_{yaw} proved to be more challenging. First, the (x, y) contour values were extracted from the **detection_tracked.txt**, then the 2D image points $(X_{\text{image}}, Y_{\text{image}})$ were calculated. The X_{image} point was calculated by averaging the x values of the contour points for each vehicle detected, while the Y_{image} point was calculated by finding the y value of the contour point closest to the ground. By doing so, we ensure the Y_{image} indicates the actual position of the vehicle on the road. Then, these 2D image points $(X_{\text{image}}, Y_{\text{image}})$ were converted into a normalized 3D camera coordinate system point $(X_{\text{c_normalized}}, Y_{\text{c_normalized}}, 1)$, to project the 2D point $(X_{\text{image}}, Y_{\text{image}})$ into 3D space along the camera line of sight. This conversion was done using the **camera1_intrinsic_calibration.dat** file located in the **calib_data** folder found on the PREVENTION website. This file contains the intrinsic camera parameters as $[fx, sk, cx, fy, cy]$, which can be used for transformations between the image plane and the camera reference system. The **camera1_extrinsic_calibration.dat** was also located in the **calib_data** folder, which holds the extrinsic calibration parameters. These parameters are provided as a 3D rotation matrix R and translation vector t needed to build the transformation matrix shown in Eq 2.

$$T = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix} \quad (2)$$

By utilizing the inverse of this transformation matrix, the camera origin $(0, 0, 0)$ and the point $(X_{\text{c_normalized}}, Y_{\text{c_normalized}}, 1)$ were converted from camera reference system to LiDAR reference system, now referred to as $P_{\text{LiDAR_Origin}}$ and $P_{\text{LiDAR_Point}}$. We were then able to calculate the vector $V_{\text{LiDAR_Direction}}$ by subtracting the points $P_{\text{LiDAR_Point}}$ and $P_{\text{LiDAR_Origin}}$. Finally, the intersection between the ray, obtained from the point $P_{\text{LiDAR_Origin}}$ and the vector $V_{\text{LiDAR_Direction}}$, and the ground plane in LiDAR reference system was calculated to find the point $(X_{\text{LiDAR}}, Y_{\text{LiDAR}}, Z_{\text{LiDAR}})$ on the plane. This point corresponds to the 3D location of the original image point $(X_{\text{image}}, Y_{\text{image}})$ in the LiDAR reference system. The X_{LiDAR} and Z_{LiDAR} represent the lateral D_{lateral} and longitudinal distance $D_{\text{longitudinal}}$, respectively. Although the maximum range detected by LiDAR is 100 meters, in our case, this number is reduced to approximately 85 meters limiting the $D_{\text{longitudinal}}$. This reduction is because of the first tilted down layer in HDL-32E, which has an elevation angle of -1.33 and intersects the ground at approximately 85 meter [17] as illustrated in Fig. 2. The last feature used to train the prediction model is the angle ψ_{yaw} , computed by measuring the angle between the forward direction of the LiDAR and the target vehicle's position using X_{LiDAR} and Z_{LiDAR} .

III. LEARNING MODEL

Deep learning has proven powerful in various complex tasks, often matching with or even surpassing human per-

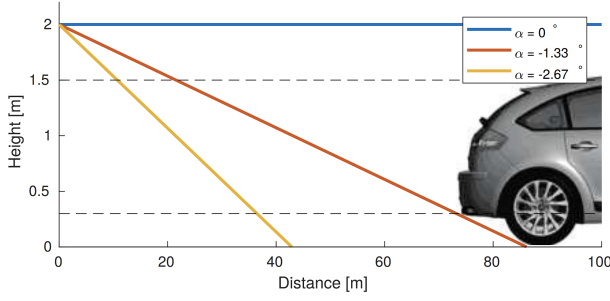


Fig. 2: LiDAR detection range without pitch variations. [17]

formance [12]. RNN is no different than any powerful deep learning model, due to its ability to learn patterns from sequential data and capture the temporal relationships between them. Over the past years, many RNN models were proposed for vehicles' behaviour prediction according to [13]. In an RNN, both previous time steps and current inputs influence the decision at the current time step, which makes it ideal for tasks that have a strong relation between time steps. Even though RNNs excel with short term sequences, when it comes to longer sequences they can suffer from vanishing or exploding gradients. Exploding gradients occur when the values used in updating the weights become excessively large during back-propagating through the network layers. In contrast, vanishing gradients happens when these update values rapidly shrink to zero making the model unable to learn effectively.

The Long Short-term Memory (LSTM) solved the vanishing and exploding gradient problem by introducing three gates: the forget gate, the input gate and the output gate. An LSTM cell has three inputs, the cell state c_{t-1} at $t-1$, the hidden state h_{t-1} at $t-1$ and the current input x_t and two outputs, the cell state c_t at t and the hidden state h_t at t . The forget gate uses the x_t and the h_{t-1} to remove the unnecessary information from the cell state, while the input gate decides the information added to the new cell state c_t . Finally, the output gate decides the information passed to the output h_t . The equations below show the calculation process [15]:

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (3)$$

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (4)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (5)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (6)$$

$$h_t = o_t \cdot \tanh(c_t) \quad (7)$$

where $\sigma(x)$ is the sigmoid function.

As shown by equations (3),(4),(5),(6),(7), the LSTM cell state, representing the LSTM's long-term memory, does not get multiplied by weights or any other factor that would lead to its exponential increase or decrease. This enable the LSTM to store data over extended periods while maintaining the gradient within manageable bounds, making it able to

process the connection in the temporal data effectively, which is needed in our case.

A. Model Architecture and inputs

In this work a deep learning LSTM classification model for predicting the vehicle lane-changes is proposed. LSTM has been specifically chosen because of how it handles long sequential data capturing the strong connections between them. As specified earlier in the Dataset section, the input to the model is a sequence of five features $[X_{\text{centre}}, Y_{\text{centre}}, D_{\text{lateral}}, D_{\text{longitudinal}}, \psi_{\text{yaw}}]$. The way these features change with time can indicate the type of the lane change, making the LSTM well suited for this task. A sequence length of 40 has been fed to the model along each sequence label, with a label of 0 indicating a left lane change, a 1 indicating a right lane change and 2 indicating a lane keep. For lane changing sequences, the vehicle sequence starts with the beginning of the lane change frame, while lane keeping can start at any other frame provided that the sequence does not contain any lane change frames. After feature extraction from the dataset, we were able to extract 251 sequences for the left lane changing, 334 sequences for the right lane changing and 1203 for lane keeping, totalling to 1787 sequences. For consistent results in the three classes, data was balanced resulting in 251 sequences for each class. Data was also normalized for faster training and better performance. Once the sequences were ready they were fed into the LSTM model for training. The number of LSTM layers were three layers, with the first layer having 512 nodes, the second having 256 nodes and the final layer having 64 nodes. The activation function used in the first and third layers was a relu activation function. A dropout layer of rate 0.5 was added after each layer to prevent overfitting. The last layer was a dense output layer, having three nodes and a softmax activation function to assign a probability to each class. The activation function and dropout rate were chosen after experimenting, to reach the best achievable results.

B. Training and Evaluation

After preparing our sequences and designing our model, we were ready to start the training process. Data was split randomly into 80% training and 20% validation. The learning rate tuning parameter was set to 0.001, the batch size was set to 128 and finally the number of epochs were set to 500. Those numbers were decided on after testing different values, some of these experiments are shown in the next section "Experiments and Results". The optimizer used was Adam optimizer and the loss function utilized was categorical cross-entropy. The model was trained for approximately 27 minutes on an AMD Ryzen 9 5900HS CPU at 3.30 GHz. The metrics used to validate the model performance were accuracy, precision and recall. Accuracy is one of the traditional metrics that judge the overall model performance, but it can be misleading when evaluating performance of imbalanced classes [18]. Therefore, precision and recall are also key performance metrics for a more accurate representation of the model performance. In our case, classes were balanced before they were given

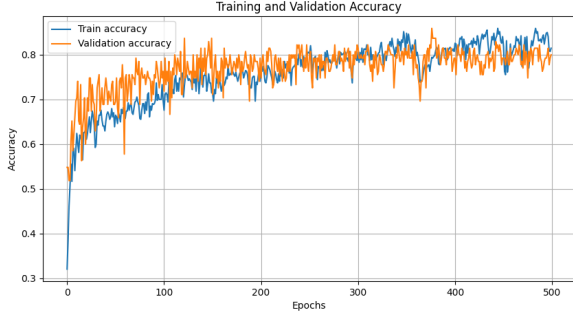


Fig. 3: Model Accuracy during Training and Validation

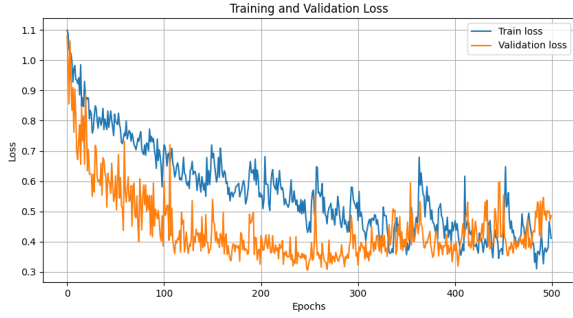


Fig. 4: Model Loss during Training and Validation

to the model, making accuracy a reliable measurement of performance. The below figures 3 and 4, show accuracy and loss of our model during training and validation and Table I presents the results of testing the model across different classes.

As seen in Table I, lane keeping exhibits the lowest performance across all three validation metrics. This may happen due to some vehicles displaying lane-change-like movements without actually changing lanes, potentially confusing the model.

IV. EXPERIMENTS AND RESULTS

Parameters tuning is a crucial step when designing a deep learning model, therefore experimenting thoroughly with these parameters is essential to achieve the best model performance. In this work, we will focus on testing the learning rate, batch size and sequence length parameters. Accuracy, recall and precision, rounded to two decimal points, will be used to evaluate our model's performance with each parameter change. Models with the highest average test results across these metrics, are the ones selected for comparison. This ensures that the model performs consistently well across all metrics.

A. Sequence length

Sequence length refers to the number of time-steps you want to process. In the context of predicting lane changes, the larger the sequence length the more data we have about the lane change event, but larger sequences are computationally expensive and may introduce data irrelevant to the lane change.

TABLE I: Model's Accuracy, Precision and Recall Metrics by Class

Class	Accuracy	Precision	Recall
Left Lane Change	0.76	0.92	0.79
Right Lane Change	0.77	0.88	0.81
Lane Keeping	0.73	0.81	0.71

Sequence Length	Accuracy	Precision	Recall
10	0.72	0.75	0.7
20	0.76	0.79	0.72
30	0.77	0.79	0.74
40	0.81	0.82	0.77
50	0.74	0.80	0.71

TABLE II: Accuracy, Precision and Recall for Different Sequence Lengths

Furthermore, the longer the sequence the less data points will be available to train the model on, while shorter sequence might not have enough context for the model to capture the sequence pattern during training. Choosing the best sequence length affects the flow of training and the test results. Five model with batch size 32, learning rate 0.0005 and five different sequence lengths were observed. All Parameters were fixed across the models for a fair comparison, ensuring these differences are only caused by the varying sequence lengths. Table II demonstrates the different results for each sequence length. From these results, we can deduce that a sequence length of 40 yielded the best overall performance, achieving an accuracy of 0.81, a precision of 0.82, and a recall of 0.77.

B. Batch Size

Batch size corresponds to the numbers of data points the model passes through before updating its weight and biases. The larger the batch size, the faster the training but this may lead to poor accuracy, while the smaller the batch size the better the accuracy and the slimmer the chances of overfitting. While small batch sizes may seem favourable, smaller batch sizes decrease the computational efficiency. Finding the optimal balance between the smaller batch sizes for their better accuracy and the larger batch size for their computational efficiency, is a task that requires a lot of trial and error. In this work, we examined three different models with 32, 64 and 128 batch sizes and observed their performance. The sequence lengths was set to 40, which achieved the best outcomes as shown in Table II, and the other parameters were fixed for comparable results. Table III shows the influence of each batch size on accuracy, precision and recall.

The results indicate that the performance of the three models is nearly identical, with the 128 batch size model being slightly better. For the upcoming learning rate experiments, we have chosen to proceed with the 128 batch size for its higher results and lower overall loss.

C. Learning Rate

Finally, we tested multiple learning rates to determine the optimal parameters for the model. The learning rate decides

Batch Size	Accuracy	Precision	Recall
32	0.81	0.82	0.77
64	0.82	0.84	0.79
128	0.83	0.86	0.79

TABLE III: Accuracy, Precision and Recall for Different Batch Sizes

Learning Rate	Accuracy	Precision	Recall
0.001	0.84	0.89	0.82
0.0005	0.83	0.86	0.79
0.0001	0.76	0.84	0.66

TABLE IV: Accuracy, Precision and Recall for Different Learning Rates

the speed at which the model learns each iteration, by determining how quickly it moves toward the minimum of the loss function. While larger learning rates lead to faster weight updates, they can result in missing the minimum of the loss function. Smaller learning rates can lead to more stable weight updates, however with significantly slower progress. The goal was to find a learning rate value small enough for the model to converge as desired while still being high enough to minimize training time. Three different learning rates were tested to compare their effectiveness, while keeping all the other parameters fixed for accurate results. Table IV shows the accuracy, precision and recall for each learning rate.

After examining various learning rates, we observed that the model with a 0.001 learning rate had a remarkable advantage over the others, showing an even more noticeable improvement when compared to the 0.0001 learning rate model. All of the above experiments were crucial to the selection process that determined the final model parameters.

V. CONCLUSION AND FUTURE RECOMMENDATIONS

In this study, we proposed a deep learning model using an LSTM to predict left lane changes, right lane changes, and lane keeping behaviors on highways. The LSTM was specifically chosen for its ability to handle long sequential data effectively, preventing the vanishing or exploding gradient problems. We utilized the PREVENTION dataset to extract the sequence of input features for training the model, starting from the first frame of lane changing intention. Through experimentation with various parameter values, we found that a learning rate of 0.001, a sequence length of 40, and a batch size of 128 optimize model performance. The performance of the model was validated during testing, where it achieved an accuracy of 84%, a precision of 89%, and a recall of 82%.

This work however has some limitations, such as ignoring the inter-vehicle dependencies and not testing the model on diverse highway scenarios. Future research will focus on adding features that capture the dynamic interactions between vehicles to enhance the model's accuracy. Moreover, we plan to test the model across various highways to develop a more versatile system.

REFERENCES

- [1] W. H. Organization, *Global status report on road safety 2023*. World Health Organization, 2023.
- [2] C. Ma and D. Li, "A review of vehicle lane change research," *Physica A: Statistical Mechanics and its Applications*, vol. 626, p. 129060, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378437123006155>
- [3] W. Zhang, Y.-H. Huang, M. Roetting, Y. Wang, and H. Wei, "Driver's views and behaviors about safety in china—what do they not know about driving?" *Accident Analysis Prevention*, vol. 38, no. 1, pp. 22–27, 2006. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0001457505001144>
- [4] L. Li, W. Zhao, C. Xu, C. Wang, Q. Chen, and S. Dai, "Lane-change intention inference based on rnn for autonomous driving on highways," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 6, pp. 5499–5510, 2021.
- [5] S. Lefevre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *Robomech Journal*, vol. 1, 07 2014.
- [6] G. S. Aoude, B. D. Luders, K. K. Lee, D. S. Levine, and J. P. How, "Threat assessment design for driver assistance system at intersections," in *13th international ieee conference on intelligent transportation systems*. IEEE, 2010, pp. 1855–1862.
- [7] C. Laugier, I. E. Paromtchik, M. Perrollaz, M. Yong, J.-D. Yoder, C. Tay, K. Mekhnacha, and A. Nègre, "Probabilistic analysis of dynamic scenes and collision risks assessment to improve driving safety," *IEEE Intelligent Transportation Systems Magazine*, vol. 3, no. 4, pp. 4–19, 2011.
- [8] M. Althoff, O. Stursberg, and M. Buss, "Model-based probabilistic collision detection in autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 10, no. 2, pp. 299–310, 2009.
- [9] M. Brand, N. Oliver, and A. Pentland, "Coupled hidden markov models for complex action recognition," in *Proceedings of IEEE computer society conference on computer vision and pattern recognition*. IEEE, 1997, pp. 994–999.
- [10] M. Liebner, M. Baumann, F. Klanner, and C. Stiller, "Driver intent inference at urban intersections using the intelligent driver model," in *2012 IEEE intelligent vehicles symposium*. IEEE, 2012, pp. 1162–1167.
- [11] G. Agamennoni, J. I. Nieto, and E. M. Nebot, "A bayesian approach for driving behavior inference," in *2011 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2011, pp. 595–600.
- [12] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-dujaili, Y. Duan, O. Al-Shamma, J. I. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: concepts, cnn architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:232434552>
- [13] S. Mozaffari, O. Y. Al-Jarrah, M. Dianati, P. Jennings, and A. Mouzakitis, "Deep learning-based vehicle behavior prediction for autonomous driving applications: A review," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 1, pp. 33–47, 2022.
- [14] O. Laimona, M. A. Manzour, O. M. Shehata, and E. I. Morgan, "Implementation and evaluation of an enhanced intention prediction algorithm for lane-changing scenarios on highway roads," in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, 2020, pp. 128–133.
- [15] L. Tang, H. Wang, W. Zhang, Z. Mei, and L. Li, "Driver lane change intention recognition of intelligent vehicle based on long short-term memory network," *IEEE Access*, vol. 8, pp. 136 898–136 905, 2020.
- [16] R. Izquierdo, A. Quintanar, I. Parra, D. Fernández-Llorca, and M. A. Sotelo, "The prevention dataset: a novel benchmark for prediction of vehicles intentions," in *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, Oct 2019, pp. 3114–3121.
- [17] R. Izquierdo, I. Parra, C. Salinas, D. Fernández-Llorca, and M. A. Sotelo, "Semi-automatic high-accuracy labelling tool for multi-modal long-range sensor dataset," in *2018 IEEE Intelligent Vehicles Symposium (IV)*, 2018, pp. 1786–1791.
- [18] I. Teeti, S. Khan, A. Shahbaz, A. Bradley, and F. Cuzzolin, "Vision-based intention and trajectory prediction in autonomous vehicles: A survey," 07 2022.